北京科技大学
University of Science and Technology Beijing

密级：公开

题 目：计算机科学导论读书报告

作 者：赵方程

学 号：42024137

学 院：计算机与通信工程学院

2020 年 12 月

目　录

1 引　言.....1
2 对计算机科学学科的认识与理解.....2
3 现代计算机中的主要计算单元.....4
3.1 对中央处理器的认识.....5
3.1.1 预测执行(Speculative Execution).....6
3.1.2 CPU 的局限性.....9
3.2 对 GPU 的认识.....10
3.2.1 通用计算中的 GPU.....11
4 结　论.....13
参考文献.....15

1 引　言

随着图灵机的诞生，通用计算理论的可行性被证明，固有的计算机设计理念被打破。通用图灵机的发展直接催生了冯·诺依曼结构的诞生，这又创造性地改变了计算机的设计方式和使用方式。经过数十年的发展，计算机的用途越来越多，使用场景也越发多样，根据不同的需求发展出了在性能、功耗、价格方面有很大不同的种类，如：个人计算机 PC, 服务器 Servers, 嵌入式计算机 Embedded Computer, 超级计算机 Super Computers。同时，计算机的使用门槛也越来越低，上限越来越高，计算机科学与技术学科的发展也逐渐走上顶峰。智能手机、自动驾驶、人类基因组计划、万维网以及搜索引擎等应用的出现与发展迅速地引领了时代潮流并且不可否认地大幅提高了社会生产力。

本读书报告将简单阐述作者对计算机科学的学科认识。

2 对计算机科学学科的认识与理解

计算机科学，是在理论上研究信息与计算，在应用上研究其实现方法的学科。曾有一个观点：计算机来源于自动化，自动化来源于机械。我认为有一定的道理，计算机科学本质上就是操作机械，只是操作的方式更为先进，用途更为广泛。所以，我认为计算机科学不仅仅是包括上述的，如何描述编程语言以用于计算以及它的应用，即用编程语言编写程序以解决某些仍未解决的实际问题，还应该包括计算机中机械的部分——计算机的组成原理等。故，私以为，计算机科学的精髓在于软件与硬件的结合。Modern computer technology requires professionals of every computing specialty to understand both hardware and software.[1]

在目前 FPGA（Field Programmable Gate Array）及 ASIC（Application Specific Integrated Circuit）出货量快速增长的现实下，计算机的发展离不开软件与硬件的深层次结合。有效且合理地使用特定的硬件运行特定的软件往往可以取得事半功倍的效果。譬如，近年来计算机科学的发展热门来到了模式识别与人工智能领域，这离不开算法的支持，也需以强大的计算能力和硬件的特性为基础。明显地，由图 2-1 可以看到，在人工智能芯片的发展中 GPU, FPGA, ASIC 占据了重要地位。

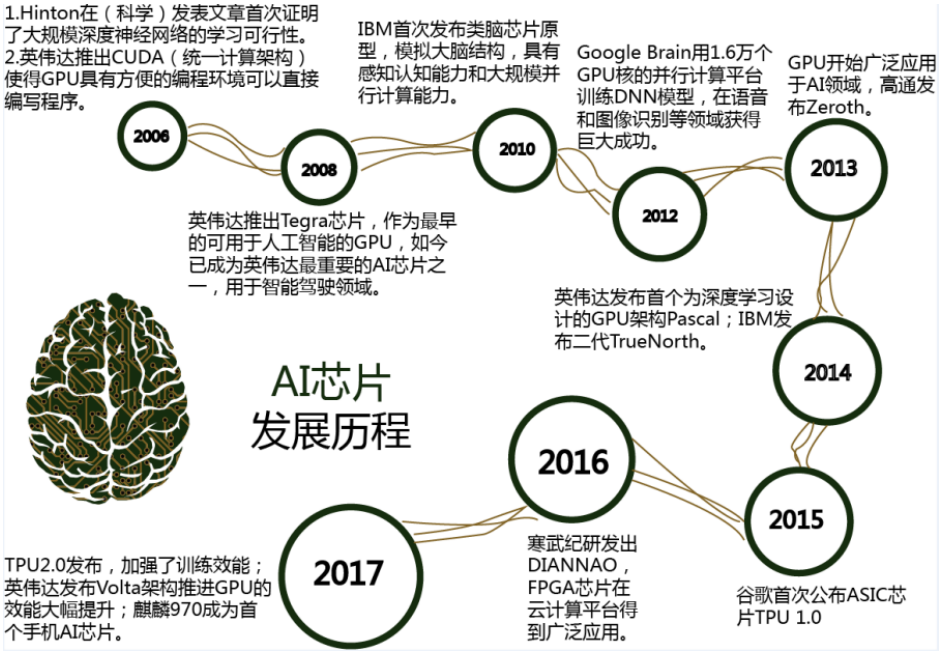


图 1-1 人工智能芯片的发展（引用自 Aminer 研究报告第十四期）

本报告第三部分将较为微观地讨论现代计算机在硬件层面的核心部件CPU 和 GPU 的发展并对未来的前景做出展望。

3 现代计算机中的主要计算单元

目前，承担计算机中主要的数据处理任务的部件是 CPU 和 GPU。中央处理器(Central Processing Unit）是现代计算机的核心部件，负责计算机系统主要的逻辑运算以及对系统其他部件，如 I/O, GPU, Memory 的统筹控制。GPU 则常常负责显示信号的输出和大量重复运算。

本章节将粗略地探讨 CPU 及 GPU 的差异。

3.1 对中央处理器的认识

现代 CPU 的结构非常复杂，但最主要的部件仍是运算器(Arithmetic logic unit),控制器(Control Unit)

自中央处理器出现以来，更高的性能极限，更低的功耗以及尽可能低的成本就是 CPU 厂商不断追求的。经历了从继电器、真空管到晶体管的飞跃后，CPU 的性能、可靠性大幅度地提升了。为了达到更高的性能，在不脱离冯 •诺依曼结构的情况下，根据 CPU 运行的不同阶段（图 3-1），厂商设计出了流水线(Instruction Pipeline)等用于加快通过指令。

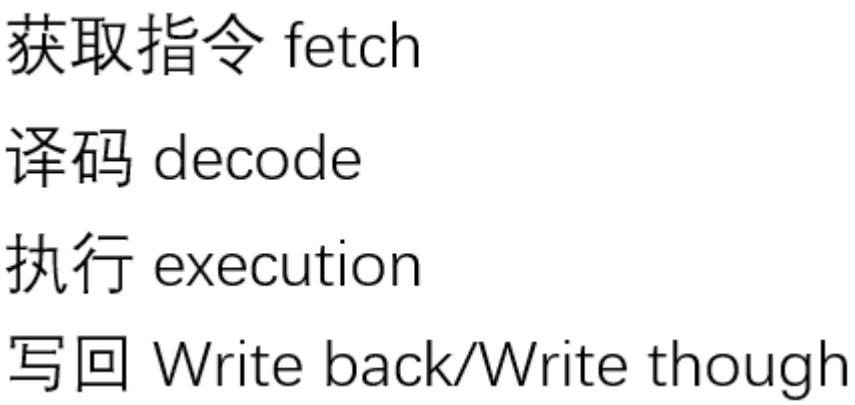


图 3-1 CPU 运行的不同阶段

同时，随着处理信息量的增加，CPU 也由最初的 4 位发展成如今的 64 位；针对并发(concurrent)或并行(parallel)的场景提供了多核心，超线程。并且，随着半导体制作工艺的进步，晶体管的物理尺度不断变小，CPU 常常以 TICK-TOCK（指一年提高制程工艺一年提高工作频率）的模式发展。

发展到如今，X86 架构下的 CPU 已经可以拥有高达 64 核心、128 线程的超大规模，时钟频率可以达到 4-5GHz，为复杂的工程计算、科学计算奠定了强有力的基础。

3.1.1 预测执行(Speculative Execution)

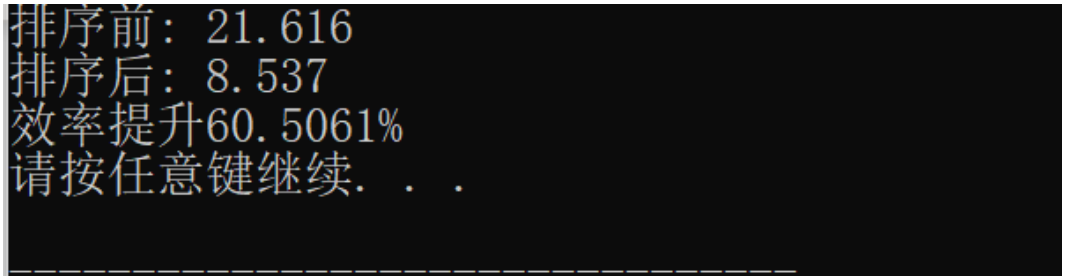
CPU 的时钟周期是非常短的，指令流水线（Instruction Pipeline）中的一个堵塞（或称之为气泡）就会导致时钟周期被浪费。CPU 采取了乱序执行(Out-of-order Execution)及预测执行等等手段以减少时钟周期的浪费。Pipeline disruption reduces the effective instruction throughput by introducing extra delays in the pipeline, since branches constitute a large portion of all the executed instructions, the efficiency of handling branches is important.[2] 处理器按照输入数据和执行单元的可用性所决定的顺序执行指令[3],而不一定按原始的顺序执行。预测执行的本质在于提前执行指令而不管这些指令是否确实需要被执行，尤其是在分支指令中，后续执行分支的跳转与否取决于该分支指令的执行结果时。如果待该分支指令经过 fetch, decode, execution 之后再执行下一条指令的 fetch, decode……那么在 CPU 的流水线上就会产生气泡，几个 CPU 时钟周期将会被浪费。对于预测执行来说，如果提前执行的指令恰好就是分支指令确定的那条指令，那么处理器就能提高工作效率。但是不可避免的是，没有根据的预测必然会造成预测的准确率低下，如果预测失败，则从 fetch 到 execution 的时间都会被浪费掉，在采用超长流水线设计的现代处理器中可能就要损失数十个时钟周期。预测执行在现代高性能处理器中被广泛地使用，为了规避预测失败的风险，处理器中需要引入一部分的电路作为分支预测器。

分支预测器(Branch Predictor)能在分支指令执行结束之前猜测哪一路分支将会被运行，用于提高分支预测的准确性，常见的分支预测技术有静态预测、动态预测、下一行预测、饱和计数、两级自适应预测器等。目前使用最多的是动态预测，即利用分支指令发生转移的历史来进行预测。

下列测试程序很好地体现了分支预测器的作用:

```
1.  #include<bits/stdc++.h>
2.  using namespace std;
3.  int main()
4.  {
5.      double consumption0=0,consumption1=0;
6.
7.      const int length=20202;
8.      int test[length];
9.
10.     for(int t=0;t<20;t++)
11.     {
12.
13.     for (int j=0;j<length;j++)
14.         {
15.             //用随机数填充数组
16.             test[j]=rand()%256;
17.         }
18.         //记录第一次开始时间
19.         clock_t start=clock();
20.         long long sum=0;
21.
22.     for(int i=0;i<10000;i++)
23.         {
24.             for(int j=0;j<length;j++)
25.             {
26.                 //分支跳转的条件 test[j]>128
27.                 if (test[j]>=128)
28.                 sum+=test[j];
29.             }
30.         }
31.
32.     //计算未排序前进行 10000 次 loop 消耗的时间
33.     consumption0+=(double)(clock()-start)/CLOCKS_PER_SEC;
34.     Sum=0;
35.     clock_t start1=clock();
36.     sort(test,test+length);
37.     for(int i=0;i<10000;i++)
38.         {
39.             for(int j=0;j<length;j++)
40.             {
41.                 //分支跳转的条件 test[j]>128
42.                 if (test[j]>=128)
43.                 sum+=test[j];
44.             }
45.         }
46.
47.     //计算排序后进行 10000 次 loop 消耗的时间
48.     consumption1+=(double)(clock()-start1)/CLOCKS_PER_SEC;
49.
50.     }
51.
52.     //输出两次运算时间并计算提升的效率
53.     cout << "排序前: "<<consumption0 << endl;
54.     cout << "排序后: "<<consumption1 << endl;
55.     cout<<"效率提升"<<100*(consumption0-consumption1)/consumption0<<"%\n";
56.     System("pause");
57.     return 0;
58. }
```

该程序在特定的机器上的运行结果如图 3-2 所示



*运行环境为 Zephyrus 14 Ryzen 7 4800HS Windows 10 2004

图 3-2 运行结果

理想情况下，当数组 test 中存放的数据是无序时，test[i+1]是否满足跳转的条件与前一项 test[i]是否满足该条件无关，此时采用动态预测的分支预测器很难通过历史数据对下一条指令的跳转与否做出正确的判断，故运算时间长，而通过 sort 函数对数组进行排序后，动态预测就可以很好地根据 test[i]的情况来预测 test[i+1]的情况，只有当数组的元素被遍历到跳转条件的分界点时会出现一定的失误，这样运算的时间就被缩短了。

从测试得出的结果来看，就该场景而言，分支预测器和预测执行能大幅度提高处理器处理分支指令的速度，幅度达到了 60.5%。

3.1.2 CPU 的局限性

首先，CPU 的核心数量无法无限制地提高，当核心之间的通信成本高于核心数量增加带来的效益时，继续增加处理器核心并不合适。（可以类比超级计算机中处理器的互联）

其次，CPU 的通用性结构对于传统的编程计算模式非常适合，同时可以通过提升 CPU 主频来提升计算速度。[4]但对于计算量大并且不需要太多的程序指令的应用场景如深度学习而言，CPU 中大量的逻辑控制电路作用则有限。

3.2 对 GPU 的认识

不同于需要进行复杂的逻辑控制、分支跳转，面向通用计算的 CPU，GPU 适合数据量巨大且运算类型统一的复杂计算。体现在电路设计上，CPU 内部具有高度复杂的控制器，以及高速缓冲区等，运算器较少；GPU 则拥有大量的运算器以进行单一或特定的运算，例如对像素的着色填充操作。图 3-3 直观地体现了这一差异。

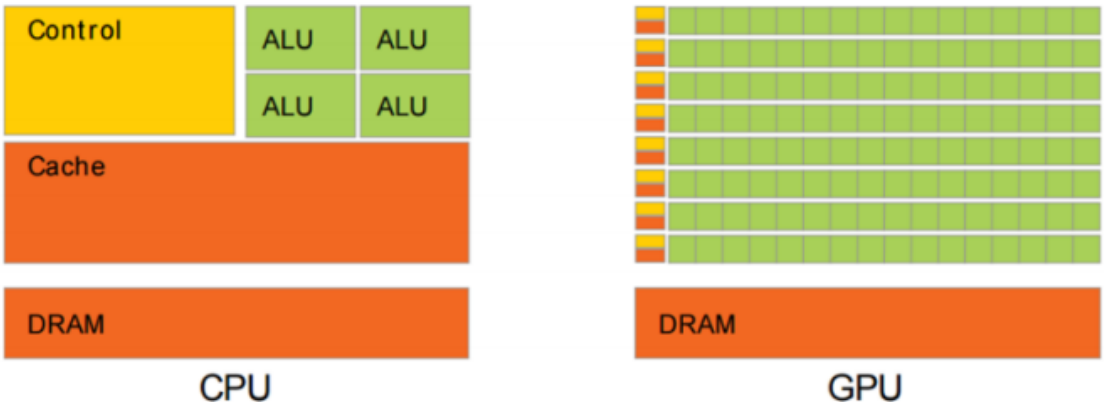


图 3-3 CPU 与 GPU 的组成 (引用自 Nvidia CUDA 文档)

尤其对于在深度学习中，无论是训练阶段或者预测阶段都大量存在的卷积运算而言，GPU 的内部结构天然地决定了其较 CPU 的速度优势。

$$\begin{matrix} 1 & 3 & 3 & 2 & 1+3 & 3+2 & 4 & 5 \\ 0 & 5+3 & 4 & = & 0+3 & 5+4 & = & 3 & 9 \\ 2 & 3 & 0 & 5 & 2+0 & 3+5 & 2 & 8 \end{matrix} \quad \text{式(3-1)}$$

$$(AB)_{ij} = \sum_{r=1}^{n} a_{ir} b_{rj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \ldots + a_{in}b_{nj} \quad \text{式(3-2)}$$

卷积运算可以转成矩阵运算，式 3-1、式 3-2 分别是矩阵加法运算和矩阵乘法运算，通过观察可知，两个 m*n 的矩阵相加可以拆分成（m*n）个加法运算，矩阵的乘法同理，一个（m*n）的矩阵和一个（n*p）的矩阵相乘可以拆分成 n 个乘法运算。卷积运算的这一可并行化的特性使其可以很好的利用 GPU 的超多计算单元，大大的加快了计算速度。

3.2.1 通用计算中的 GPU (General-purpose computing on graphics processing units）

目前适用于 GPU 的通用计算标准有由 Nvidia 推出的统一计算架构 CUDA (Compute Unified Device Architecture)以及 APPLE 提出的开放运算语言 OpenCL (Open Computing Language)

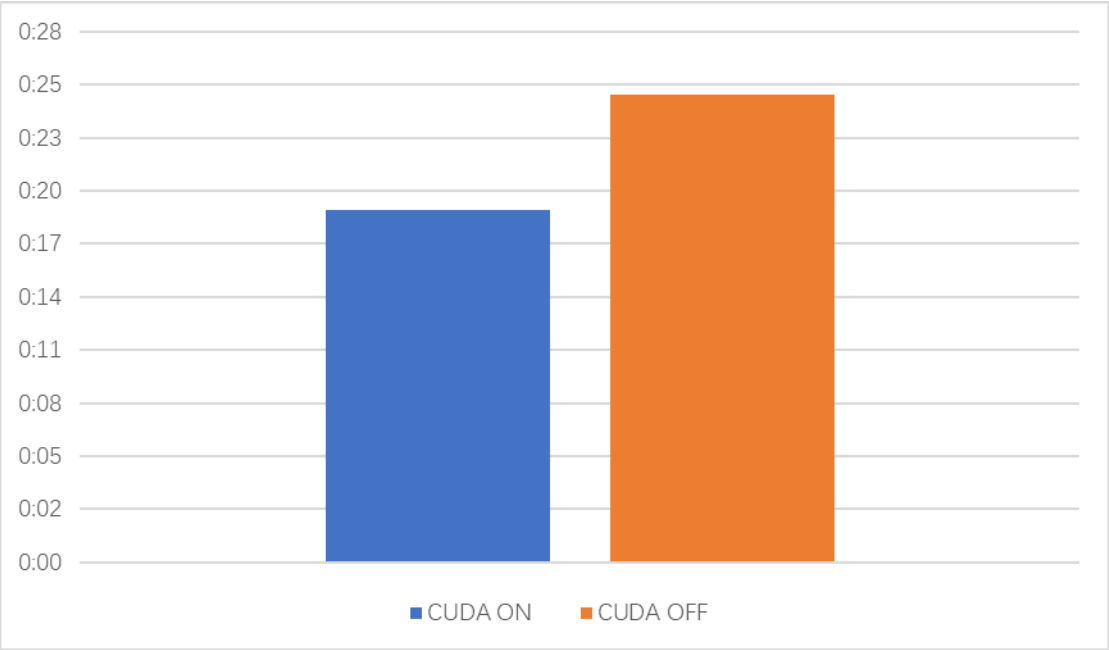
通过使用 CUDA，C/C++程序可以利用 GPU 来进行加速，CUDA 允许调用库中的函数，也可以使用 C、C++、Fortran 和 Python 等语言开发自定义应用程序。类似地，OpenCL 也是将计算密集型的任务装载到处理单元，如 GPU 上来加快应用程序的速度。

通过 Blackmagic RAW Speed Test 测试系统解码全分辨率 Blackmagic RAW 帧的速度（表 3-1）以及在 Davinci 中导出同一段 4k 视频消耗时间（图 3-4）展示了 CUDA 对应用软件的加速效果。

表 3-1 CUDA 对解码高分辨率图像的帧速率的影响

Resolution	CUDA ON	CUDA OFF
8K 3：1	24	10
8K 12：1	26	12
6K 3：1	39	16
6K 12：1	42	35
4K 3：1	100	41
4K 12：1	107	90
HD 3:1	402	166
HD 12:1	426	206

解码帧速率单位取 FPS



*运行环境为 Zephyrus 14 Ryzen 7 4800HS Windows 10 2004 with RTX 2060 max-Q

图 3-4 CUDA 对导出同一段 4K H.264 视频的加速

可见当通用计算平台被良好地搭建时，GPU 在加速应用上有巨大的优势。

4 结　论

就桌面级 CPU 而言，产品发展趋势必然是更多的核心以及更强的单核性能。然而随着工艺制程进步逐渐进入瓶颈区，单核性能的提高的难度大大增加，不再是主流；以多核心为主的发展将是不可逆的。这就对应用程序编写、优化的要求提高了。正如第二章所言，计算机科学的精髓在于软件与硬件的结合。不断提高的生产生活需求对计算机的发展有着指引作用：软件需要更全面地考虑硬件基础，硬件则需要对软件进行特定的优化。

在这一背景下 GPU 的通用计算的发展应该会逐渐走向繁盛，因为大部分场景下，对于时间复杂度而言，空间复杂度并不是特别重要，在有足够存储空间的情况下尽可能地用并行计算提高运算速度是可以接受的。

由于作者知识的局限性，暂时无法做出更为深入的探讨，文中如有错误，还望斧正！

参考文献

[1] David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface [M] Fifth Edition, Asian Edition: China Machine Press, 2013: 5.

[2] Shien–Tai Pan, Kimrning So, Joseph T. Rahmeh. Improving the Accuracy of Dynamic Branch Prediction [J] ASPLOS V: Proceedings of the fifth international conference on Architectural support for programming languages and operating systems, 1992 :76–84. https://staff.fnwi.uva.nl/s.polstra/psa2017/p76-pan.pdf

[3] Kukunas, Jim. Power and Performance. Software Analysis and Optimization [M] Morgan Kaufmann Publishers, 2015:37.

[4] 清华-中国工程院知识智能联合实验. 人工智能芯片研究报告[R/OL], 2018 : 8 https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&cad=rja&uact=8&ved=2ahUKEwjJkpei2d_tAhVoxYsBHQGYD54QFjABegQIAhAC&url=http%3A%2F%2Fpdf.dfcfw.com%2Fpdf%2FH3_AP201811131243999088_1.pdf&usg=AOvVaw0cZ4jtZtnvFG3CqHtyTXwh